Informe de Refactorización a Arquitectura MVC

Proyecto: Sistema Bancario

Objetivo: Transicionar desde una arquitectura monolítica con archivos planos a un patrón **Modelo-Vista-Controlador (MVC)**, desacoplando la lógica de negocio, la persistencia y la interfaz gráfica.

1. Estructura y Distribución de Módulos

A. Capa de Modelo (src/models/)

- **Archivos movidos:** persona.py, cliente.py, empleado.py, cuenta.py, cuenta_ahorro.py, transaccion.py
- **Justificación:** Representan las entidades de dominio y las reglas de negocio puras (validaciones de campos, cálculo de saldos, control de límites y herencia).
- **Regla de Diseño:** Cumple estrictamente con la **regla de oro del desacoplamiento**: no posee ninguna dependencia ni importación de Flet (import flet).

B. Capa de Persistencia (src/persistencia/)

- **Archivos movidos:** models.py (Mapeo ORM SQLAlchemy), dao.py, dao_cliente.py, dao_personal.py
- **Justificación:** Aisla la interacción directa con la base de datos SQLite (sistema_bancario.db). Se separaron las consultas y operaciones CRUD de las entidades puras para facilitar la prueba de la capa de modelo con bases de datos en memoria.

C. Capa de Controlador (src/controllers/)

- **Archivos movidos / organizados:** autenticacion.py, gestion_clientes.py, servicios_movimientos.py, navegacion.py
- **Justificación:** Actúa como el intermediario/orquestador entre las Vistas y los Modelos. Recibe las acciones desencadenadas por el usuario, procesa la lógica correspondiente, gestiona el flujo de navegación entre pantallas y captura las excepciones de negocio.

D. Capa de Vista (src/views/)

- **Archivos movidos:** login.py, sesion.py, home_cliente.py, home_personal.py, home_operaciones.py, home_crear_cliente.py, home_historial.py, vista_gestion_clientes.py
- **Justificación:** Mantiene únicamente el código encargado del renderizado gráfico mediante Flet. Es una capa declarativa ("tonta") que no ejecuta lógica ni consultas a la base de datos, delegando todos los eventos hacia los controladores.

E. Módulos Auxiliares y Legacy (src/legacy/ y docs/)

- **Archivos movidos a src/legacy/:** legacy_app_banco.py, legacy_datos_demo.py, legacy_exportar_pdf.py (archivos de versiones anteriores preservados como histórico).
- **Archivos movidos a docs/:** Documentos de prueba y reportes PDF (Reporte_Vane...) para mantener la raíz limpia.

2. Archivos en la Raíz del Proyecto

Se preservaron en el directorio raíz los archivos esenciales de configuración y arranque:

- main.py: Punto de entrada principal para ejecutar la aplicación.
- requirements.txt: Dependencias del proyecto.
- .gitignore y README.md: Control de versiones y documentación general.
- tests/: Carpeta con la suite automatizada de pruebas (pytest).
- venv/: Entorno virtual aislado.

3. Impacto en la Suite de Tests (pytest)

La reestructuración física requirió adaptar los enunciados de importación (imports) en la carpeta tests/ (por ejemplo, actualizando de from cliente import Cliente a from src.models.cliente import Cliente). La suite de 47 tests continúa actuando como la red de seguridad (*Safety Net*) para certificar que la migración arquitectónica no introdujo regresiones en las reglas del negocio bancario.

Actualización final de la Suite de Tests (pytest):

Se refactorizaron las importaciones internas de los 10 módulos de prueba (test_*.py) y de los módulos internos del proyecto (src/) para alinearlos a la nueva estructura por paquetes. Con esto se logró una ejecución **100% exitosa de la suite de pruebas (44 tests pass / 3 skipped)**, garantizando que el paso a la arquitectura MVC no introdujo regresiones en el sistema.

Cómo cumple mi nueva versión del proyecto Sistema Bancario" con la consigna de la actividad:

1. Capa de Modelo (src/models/ y src/persistencia/)

- **Actividad:** Albergar las clases de dominio (Persona, Cliente, Cuenta), acceso a BD (DAO) y reglas de negocio, con la restricción absoluta de **cero imports o sentencias de Flet**.
- **Cómo lo cumple el proyecto:**
    - Las clases de dominio puro quedaron separadas en src/models/ (persona.py, cliente.py, cuenta.py, cuenta_ahorro.py, transaccion.py).
    - La capa de persistencia y el ORM de SQLAlchemy quedaron guardados en src/persistencia/ (dao.py y models.py).
    - Ninguno de estos archivos tiene import flet. La mejor prueba de esto es que **todos los tests en tests/ pudieron ejecutarse y pasar en verde** en modo *headless* sin necesidad de levantar ventanas gráficas.

2. Capa de Vista (src/views/)

- **Actividad:** Contener los controles reactivos y layouts visuales de Flet (login.py, sesion.py, etc.), limitándose a renderizar datos y capturar eventos sin hacer cálculos de saldos ni consultas directas a la BD.
- **Cómo lo cumple el proyecto:**
    - Todas las interfaces gráficas están agrupadas en src/views/ (login.py, sesion.py, vista_gestion_clientes.py).
    - La interfaz solo dibuja los componentes neón y responde a eventos del usuario (como clics en botones) delegando la acción al controlador.

3. Capa de Control (src/controllers/)

- **Actividad:** Actuar como mediador orquestador que intercepta las señales de la Vista, invoca al Modelo/DAO y envuelve las operaciones en bloques try/except para manejar excepciones de negocio (como SaldoInsuficienteError).
- **Cómo lo cumple el proyecto:**
    - Los orquestadores residen en src/controllers/ (autenticacion.py, navegacion.py).
    - Estos archivos reciben las peticiones de las pantallas, llaman al DAO para realizar las consultas o transacciones, y si ocurre un error (por ejemplo, clave incorrecta o saldo insuficiente), capturan la excepción para indicarle a la Vista que muestre una alerta sin que la aplicación se caiga.